

Blockchain Assisted Trust Management for Data-Parallel Distributed Learning

Yuxiao Song^{ID}, Daojing He^{ID}, *Member, IEEE*, Minghui Dai^{ID}, Sammy Chan^{ID}, *Senior Member, IEEE*, Kim-Kwang Raymond Choo^{ID}, *Senior Member, IEEE*, and Mohsen Guizani^{ID}, *Fellow, IEEE*

I. INTRODUCTION

Abstract—Machine learning models can support decision-making in mobile terminals (MTs) deployments, but their training generally requires massive datasets and abundant computation resources. This is challenging in practice due to the resource constraints of many MTs. To address this issue, data-parallel distributed learning can be conducted by offloading computation tasks from MTs to the edge-layer nodes. To facilitate the establishment of trust, one can leverage trust management, say to use trust values derived from local model quality and evaluations by other nodes as access criteria. Nonetheless, security and performance considerations remain unsolved. In this paper, we propose a blockchain-assisted dynamic trust management scheme for distributed learning, which comprises nodes attributes registration, trust calculation, information saving, and block writing. The proof of stake (PoS) consensus mechanism is leveraged to enable efficient consensus among the nodes using trust values as stakes. The incentive mechanism and corresponding dynamic optimization are then proposed to further improve system performance and security. The reinforcement-learning approach is leveraged to provide the optimal strategy for nodes' local iterations and selection. Simulations and security analysis demonstrate that our proposed scheme can achieve an optimal trade-off between efficiency and quality of distributed learning while maintaining system security.

Index Terms—Blockchain, distributed learning, trust management, and reinforcement learning.

Received 26 June 2024; revised 5 November 2024; accepted 9 December 2024. Date of publication 23 December 2024; date of current version 4 April 2025. This work was supported in part by the National Key R&D Program of China under Grant 2021YFB2700900, in part by the National Natural Science Foundation of China under Grant 62376074, in part by Shenzhen Science and Technology Program under Grant KJZD20231023095959002, Grant SGDX20230116091244004, Grant KJZD20230923114405011, Grant JSG-GKQTD20221101115655027, Grant KCXST20221021111404010, and Grant JSGG20220831103400002, in part by the Fundamental Research Funds for the Central Universities under Grant HIT.OCEF.2024047, and in part by the Fok Ying Tung Education Foundation of China under Grant 171058. The work of Kim-Kwang Raymond Choo was supported only by the Cloud Technology Endowed Professorship. Recommended for acceptance by L. Kong. (*Corresponding author: Daojing He.*)

Yuxiao Song and Daojing He are with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, China (e-mail: 23b351007@stu.hit.edu.cn; hedaojinghit@163.com).

Minghui Dai is with the School of Computer Science and Technology, Donghua University, Shanghai 201620, China (e-mail: minghuidai@dhu.edu.cn).

Sammy Chan is with the Department of Electrical Engineering, City University of Hong Kong, Hong Kong, China (e-mail: eeschan@cityu.edu.hk).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, University of Texas at San Antonio, San Antonio, TX 78249-0631 USA (e-mail: raymond.choo@fulbrightmail.org).

Mohsen Guizani is with the Machine Learning Department, Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates (e-mail: mguizani@gmail.com).

Digital Object Identifier 10.1109/TMC.2024.3521443

THE rapid development of the beyond 5G (B5G)/6G networks and machine learning (ML) have supported various advanced ML applications, such as mobile target tracking, intelligent patrol systems, and intelligent transportation [1]. Mobile terminals (MTs) generate and collect huge amounts of data for ML model training to meet the requirements of advanced ML applications, necessitating significant computing resources. However, conducting such model training is challenging for resource-constrained MTs. This reinforces the importance of designing effective solutions that support the migration of ML model training workload across edge-layer nodes, including those with sufficient computing resources that are in close proximity to MTs [2], [3].

One effective method for training ML models on large datasets is to leverage a data-parallel processing framework [4], [5]. Utilizing this approach, the MTs' datasets are partitioned and distributed across multiple edge-layer nodes, and each node utilizes the pre-trained ML model to execute local training on its portion of the dataset and further uploads the locally trained model to the server to generate the aggregated model [6]. The generated global model is then downloaded to the MTs to enable ML applications. Therefore, in this approach, the MTs serve as both data owners and model users, while the edge-layer nodes act as the model trainers. This enables parallel processing of the training data, which can significantly reduce the overall training latency and mitigate the computational and memory limitations of individual MTs by effectively utilizing the computing resources of the edge-layer nodes. However, because of the complexity and variability of the wireless environment, the edge-layer nodes are vulnerable to hackers or malicious users, which leads to security risks [7].

To mitigate or reduce the risk of nodes behaving maliciously, we can introduce a trust management system [8], [9], [10]. Such a system can help us to distinguish whether the nodes are trustworthy and eligible to participate in the computation group. Besides, when employing edge-layer nodes for data-parallel distributed learning, the nodes may generate low-quality locally trained model due to insufficient effort, consequently affecting the convergence of decentralized learning. Moreover, the training process involves multiple rounds, which requires continuous evaluation of nodes' generated models and dynamic trust management to identify the malicious nodes during the model training process before model aggregation, instead of

relying on static authentication schemes. In this work, to address these challenges, we aim to develop dynamic and efficient trust management scheme tailored for data-parallel distributed learning in wireless environment. Specifically, the quality of nodes' local models, the evaluation by other nodes, as well as the weighted sum of the historical behaviors of the node, are utilized to calculate the nodes' trust values to act as the criteria for nodes access during the learning process.

In this work, we explore and demonstrate the potential of using blockchain [11], [12] to address the shortcomings of centralized authorization and provide a secure and efficient blockchain assisted trust management scheme for data-parallel distributed learning in wireless environment. Specifically, our proposed blockchain assisted trust management scheme supports nodes attributes registration, trust calculation, information saving, and block writing. Each node is required to register its attribute to the attribute blockchain to verify the eligibility of nodes' attributes and identities. Besides, we introduce the threshold of the trust value to allow those nodes with sufficient trust to participate in the computation group. Those nodes remaining in the computation group are eligible to store their computation results and trust values in the blockchain. In addition, the proof of stake (PoS) [13] is utilized as the blockchain's consensus mechanism. It enables efficient consensus among the nodes in the computation group, with the trust value serving as the stake. Nodes with larger stakes are more likely to be elected as the server to organize blockchain assisted trust management. Also, blockchain-based decentralized distributed learning removes the need for a central server for authentication and model aggregation; thus, reducing the risk of a single point of failure. Additionally, each global iteration includes T rounds of trust management and updates to the computation group based on the trust values, enabling continuous evaluation of nodes' model training. Only the local ML models of those nodes that remain in the computation group after these T rounds of blockchain-assisted trust management are packed and sent to the server for aggregation.

Nevertheless, executing local model training and improving a node's trust value may incur additional energy consumption and latency. Therefore, we design an incentive mechanism to motivate nodes to invest effort in the learning process while discouraging any malicious behaviors, ultimately leading to improved overall system performance and security level. The quantitative relationship between the number of local trainers' iterations and the accuracy of the locally trained models serves as a more fine-grained and effective method to reflect nodes' behaviors for calculating trust and determining incentive rewards. Furthermore, we investigate dynamic optimization for the node's local iterations and the threshold of the trust value to maximize the overall utility of the nodes within the process of distributed learning. We model the optimization as a finite-horizon Markov decision process (MDP) [14] and propose a proximal policy optimization (PPO) [15] based deep reinforcement learning (DRL) [16] approach to solving it. This provides the optimal strategy of the nodes' local iterations and selection, which achieves an optimal trade-off between efficiency and quality of distributed learning.

In summary, the main contributions of this article are as follows.

- *Data-Parallel Distributed Learning Framework:* We propose a data-parallel distributed learning framework in which data from MTs and model training tasks are off-loaded to edge-layer nodes. Specifically, a leader terminal with sufficient storage capacity aggregates datasets from other MTs to create a main dataset, which is then transmitted to the edge-layer nodes for model training. The elected server in the edge layer distributes the pre-trained model and the sub-datasets, obtained by splitting the main dataset, to each edge-layer node for local model training. The locally trained models are then aggregated into a global model, which is downloaded to the MTs to enable ML applications. In this framework, the MTs serve as both data owners and model users, while the edge-layer nodes act as the model trainers.
- *Blockchain assisted Trust Management Scheme:* We propose a blockchain assisted trust management scheme for data-parallel distributed learning to enable continuous evaluation of edge-layer nodes' model training and establishes a dynamic join and exit mechanism for the nodes. The scheme supports nodes attributes registration, trust calculation, information saving, and block writing. A node's trust value is calculated based on the quality of its local model, the evaluation by other nodes, as well as the weighted sum of its historical behaviors to determine its access eligibility. By utilizing trust values derived from nodes' behaviors, we introduce a PoS mechanism that leverages these trust values as stakes. This approach enables the effective selection of the server and nodes to participate in the computation group. Besides, the MaxClique method-based filtering process and punishment mechanism are employed to identify and filter out the nodes engaged in malicious behavior. Thus, our proposed scheme effectively addresses nodes' misbehavior, prevents system performance degradation, and ensures the permanent recording of computation results.
- *Incentive-Oriented Problem Formulation for Optimizing Nodes Contributions and DRL-based Algorithm Design:* To motivate the nodes to invest effort in the learning process while discouraging any malicious behaviors, we design an incentive mechanism and investigate dynamic optimization for the node's local iterations and the threshold of the trust value to maximize the overall utility of the nodes. By applying DRL technology to solve it, we can derive the optimal strategy for nodes' local iterations and selection, which achieves the trade-off between learning efficiency and quality, while providing better utility compared to other benchmark schemes.
- *Performance Evaluation and Security Analysis:* Experimental results are provided to demonstrate the performance of our proposed scheme under different parameter settings in comparison with other baseline schemes. Security analysis is also provided to show that our proposed scheme is capable of effectively tackling common security threats,

such as poisoning attacks, dishonest trust calculations, and free-riding attack.

The remainder of this article is organized as follows. We present the literature review in Section II. In Section III, we present the system model for the data-parallel distributed learning. Section IV presents the design of blockchain assisted trust management for distributed learning. In Section V, we present the optimization problem formulation. In Section VI, a PPO based algorithm is proposed to effectively solve the optimization problem. We present the experimental results and the security analysis in Section VII to show the performance advantages and the reliability of our proposed scheme. Section VIII concludes our work and discusses future direction.

II. LITERATURE REVIEW

A. Blockchain for Distributed Learning

Because of the various characteristics like decentralization, public ledger and immutability, several works have focused on integrating blockchain and edge computing to provide secure and efficient data sharing. In [2] and [3], the authors proposed blockchain-based cooperative computation offloading in vehicular edge computing networks to ensure secure data migration and synchronization. Thus, integrating blockchain with distributed learning creates a trustworthy and reliable distributed learning paradigm. In [17], Qu et al. proposed a blockchained federated learning (FL) architecture for decentralized Big Data-driven cognitive computing, in which the blockchain's verification mechanism ensures data authenticity while selecting trustworthy devices. The proof of work (PoW) mechanism has been adopted in [17] to generate the block. However, PoW demands solving complex cryptographic puzzles, causing high costs and making it unsuitable for energy-constrained nodes. In [18], Putra et al. proposed a blockchain-based parameter aggregation technique to enhance the trustworthiness in FL by utilizing a proof of authority (PoA) consensus mechanism. However, PoA relies on a limited set of pre-approved validators, introducing centralization and risks of single point of failure and potential collusion that could compromise the reliability of the blockchain. In [19], Jiang et al. integrated FL with directed acyclic graph (DAG) blockchain to develop an energy-efficient, decentralized, and scalable digital twin-driven wireless network framework. However, DAG lacks support for model evaluation and is unsuitable for trust management. Thus, a more efficient and robust consensus mechanism is needed among nodes to support blockchain assisted trust management for distributed learning.

To improve the efficiency and model accuracy of the blockchain assisted distributed learning, in [20], Xu et al. proposed an asynchronous FL that utilizes blockchain to enhance reliability. Specifically, a committee-based consensus algorithm with minimal communication latency has been designed to periodically elect a new leader to identify poor-quality local models and perform model aggregation. Nevertheless, the proposed scheme in [20] lacks the attribute registration mechanisms to prevent nodes from adopting multiple identities or using whitewashed identities to join the system. This makes the system vulnerable to sybil attacks or newcomer attacks. Besides, the

trade-off between model performance and training efficiency has not been discussed in the aforementioned literature.

To motivate the nodes to actively participate the distributed learning, some works have adopted incentive mechanism for blockchain-based distributed learning. In [21], Zhu et al. proposed an incentive mechanism for blockchain-based FL that rewards users based on the data they share. This mechanism tracks users' contributions to the global model, ensuring that all participants are treated fairly. However, the scheme proposed in [21] lacks consideration of the computational capabilities users should contribute to local model training. To address this issue, in [22], Zhang et al. proposed an incentive mechanism for secure blockchain-based FL in IoT systems, which rewards devices based on their computing power and encourages capable devices to train and share models. In [23], Tang et al. proposed an incentive mechanism for blockchain-empowered FL that motivates efficient cooperation among users. Specifically, the organization uses users' processing capacity and unit monetary transfer per training round to calculate the compensation for the training costs. However, the quantitative relationships between the number of trainers' local iterations (a key parameter of model training) and the accuracy of the locally trained model, along with the corresponding system costs, have not been considered in [22], [23].

B. Trust Management for Distributed Learning

Trust evaluation can effectively reflect the reliability of nodes' behaviors. There have been several works adopting trust management for distributed learning. For instance, in [24], Gholami et al. proposed a trusted decentralized FL framework to address security threats, where the trust values are derived from the number of agents' benign and malicious behaviors using a Beta distribution model. Moreover, to ensure secure data storage, some works have introduced blockchain into trust management assisted distributed learning. In [25], Lv et al. proposed a misbehavior detection system for privacy-preserving FL in the vehicular ad hoc network, incorporating blockchain with DPoA consensus and trust assessment. Specifically, the trust values of roadside units (RSUs) are calculated based on model accuracy to incentivize RSUs to produce high-accuracy models. Besides, to further investigate the impact of users' contributed data quality on distributed learning tasks, in [26], Qi et al. proposed a blockchain-based FL with a reputation mechanism based on the model quality and users' contribution evaluation. The reputation has been used to coordinate effective data contributions and allocate rewards accordingly. To avoid relying on single trust factor, such as the quality of users' local models or interaction results, for calculating trust values in distributed learning, in [8], Guo et al. proposed a fine-grained trust evaluation model to calculate the trust values of the users involved in the FL system in digital twin for mobile networks. Specifically, users' behaviors from multiple dimensions have been utilized to conduct trust calculation, providing a more accurate representation of their trust levels. However, the user behaviors mentioned in [8] are limited to model training patterns, directly linked to the accuracy of the trained model. Due to the complexity and variability of the

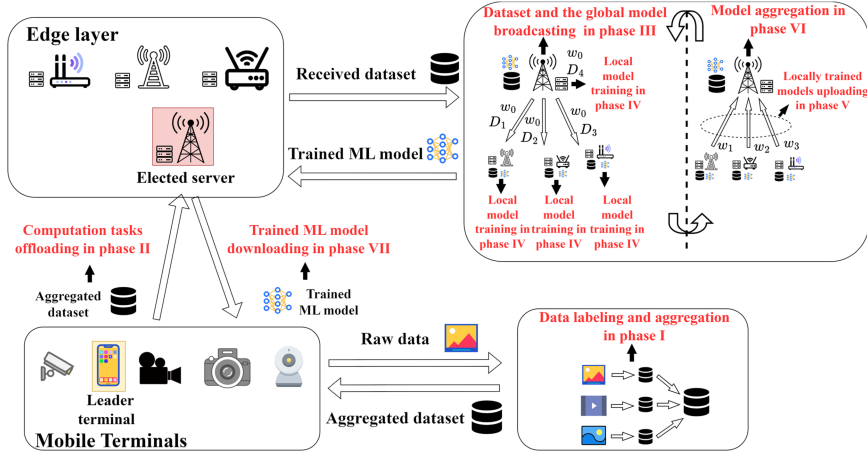


Fig. 1. System model of the proposed data-parallel distributed learning.

wireless environment, it is crucial to consider and incorporate additional dimensions of user behavior, such as illegal access and fake data, into the calculation of trust values to effectively identify malicious nodes.

Besides, the trust management methods for distributed learning mentioned above calculate users' trust values only after their locally trained models have been uploaded for model aggregation. However, as the users' model training includes multiple rounds before generating locally trained models, it is essential to conduct continuous supervision on the users' model training throughout the process of decentralized learning to identify the malicious users more timely and prevent the negative effect on the convergence of global model. Moreover, the optimization of the number of local iterations and the trust value threshold to derive the optimal strategy for nodes' local iterations and selection, as well as achieving reliable and cost-efficient model convergence, has not been considered in the above trust management schemes.

Based on these concerns, this paper presents a blockchain assisted trust management scheme for data-parallel distributed learning. PoS is adopted as the consensus mechanism due to its speed, low decentralization, mild cost, and ability to support trust-based evaluation. This scheme enables continuous evaluation of edge-layer nodes' model training and establishes a dynamic join and exit mechanism during distributed learning. Each global iteration involves multiple rounds of trust evaluations based on local model quality, peer evaluations, and a weighted sum of historical behaviors. Besides, we use the quantitative relationship between local iterations and model accuracy to provide a finer method to reflect nodes' behaviors for calculating trust and determining incentive rewards. We propose an incentive mechanism and jointly optimize the number of nodes' local iterations and the trust threshold to achieve a trade-off between efficiency and quality while ensuring security.

III. SYSTEM MODEL

A. Network Model

We consider a cluster of MTs deployed within a designated area for the purpose of conducting raw data collection. The

collected data is then labeled and transmitted to the edge layer for model training. Here, we consider that the leader terminal with sufficient storage capacity is responsible for collecting other terminals' datasets to form an aggregated dataset. Moreover, a set of edge-layer nodes, denoted as $\mathcal{I} = \{1, 2, \dots, I\}$, collaboratively execute computing tasks offloaded from MTs through multiple rounds of global iterations. An elected server is randomly appointed among the edge-layer nodes in the first global iteration round. As shown in Fig. 1, the operations of the data-parallel distributed learning can be described as follows.

- *Phase I to Phase II - Data labeling and aggregation and computation tasks offloading:* The raw datasets collected by MTs are labeled based on specific phenomena or publicly available datasets. Subsequently, the leader terminal aggregates them to generate a comprehensive dataset. The aggregated dataset is then transmitted to the elected server in the edge layer for model training.
- *Phase III - Dataset and the global model broadcasting:* The received dataset by the elected server is partitioned into subsets to ensure that the resulting sub-datasets are independent and identically distributed (IID). The sub-dataset D_i^{train} and the pre-trained global model ω_0 are then broadcasted to each node.
- *Phase IV - Local model training:* In this phase, each node executes the local ML model training based on the sub-dataset and the pre-trained model assigned to it.
- *Phase V - Locally trained models uploading:* After completing local model training, the behaviors of the nodes are evaluated to determine whether the nodes can remain within the computation group. The nodes that successfully remain in the computation group upload their local models ω_i . Besides, the server is re-elected within the computation group.
- *Phase VI - Model aggregation:* After receiving the nodes' local models, the server performs the model aggregation to generate the aggregated model. The aggregated model is then broadcasted to the nodes within the computation group, as illustrated in Phase III. The process of global iteration includes Phase III through Phase VI and is required

to be executed for a predefined number of rounds denoted as N .

- *Phase VII - Trained ML model downloading:* After N rounds of global iteration, the trained model is then downloaded to MTs to facilitate the ML-enabled applications.

B. Computation Model

We consider that the sub-dataset allocated to node i to train its model is represented as D_i^{train} . The system cost of node i 's model training includes both energy consumption and latency. Based on [27], the energy consumption for node i 's local iterations can be represented as

$$E_i^{\text{loc}} = \eta_i^{\text{loc}} \kappa C S(D_i^{\text{train}}) \mu_i^2, \forall i \in \mathcal{I}, \quad (1)$$

where μ_i denotes node i 's CPU's processing-rate, η_i^{loc} represents the number of node i 's local iterations, κ represents the CPU power-efficiency, C represents the number of CPU-cycle for executing one bit of data and $S(\cdot)$ represents the size of the dataset.

Besides, the latency for node i 's local iterations can be represented as

$$l_i^{\text{loc}} = \frac{\eta_i^{\text{loc}} C S(D_i^{\text{train}})}{\mu_i}, \forall i \in \mathcal{I}. \quad (2)$$

C. Threat Model

The security threats of the data-parallel distributed learning are discussed as follows.

- *Data poisoning attack and model failure poisoning attack:* In the system model of the data-parallel distributed learning, each node within the computation group is required to train its local model based on the assigned dataset. Some malicious nodes may conduct data poisoning attacks [28] via injecting noisy data or tampering with data labels. Besides, some malicious nodes may conduct model poisoning attacks [29] via uploading irrelevant or false local models to the server. These attacks can lead models to produce misclassifications, incorrect decisions, or other misleading behaviors, thus compromising the security and the performance of the system. In this article, we only consider the untargeted poisoning attacks that lead to a significant degradation in model performance [30].
- *Historical behaviors tampering attack:* Some malicious nodes may tamper, replace, or delete the historical behaviors stored in the central server with the goal of eliminating the effect of past misbehaviors on the trust value.
- *Sybil attack:* A malicious node masquerades as multiple nodes and broadcasts these multiple masquerading identities to the entire system, with the goal of gaining control of the system and manipulating the process of behaviors evaluation.
- *Newcomer attack:* Some malicious nodes attempt to rejoin the network using a new identity with the intention of maliciously removing its previously recorded bad misbehaviors and gaining a new trust value.
- *Self-Promoting attack:* Some malicious nodes attempt to perform false evaluations of their locally trained models or

make miscalculations in the process of trust calculation to obtain exaggerated values of the accuracy of their locally trained models and trust values.

- *Bad-Mouthing attack:* The calculation of trust value is based on the evaluations of other nodes. Some malicious nodes may report spurious evaluation results to degrade other nodes' trust values, which reduces the probability of other nodes joining the computation group, making it more likely for the malicious nodes to join the computation group.
- *Ballot-Stuffing attack:* Some malicious nodes may collude with other nodes to obtain the exaggerated evaluation value of their locally trained model, which upgrades their trust value.
- *Free-riding attack:* The goal of the computation group is to require each node to improve the accuracy of its local ML model to complete the collaborative computation task. However, due to the factors such as excessive latency and energy consumption, some nodes may be unwilling to conduct local iterations to improve the quality of their locally trained ML models as well as their trust values, which reduces the overall computational quality of the computation group.

It is worth noticing that in this article, we mainly focus on effectively defending against adversarial attacks [31] and cheating during trust calculation and storage that may impact the convergence speed or model accuracy in decentralized learning.

IV. BLOCKCHAIN ASSISTED TRUST MANAGEMENT FOR DATA-PARALLEL DISTRIBUTED LEARNING

A. Proposed Blockchain Assisted Trust Management Scheme

We present a blockchain assisted trust management scheme via the PoS mechanism for data-parallel distributed learning. With PoS-based consensus mechanism, the swift and scalable consensus can be achieved among the nodes without excessive system cost and latency. A node's trust value is adopted as the stake to dynamically select the server to organize the process of data-parallel distributed learning, as well as determining the criteria for selecting nodes to be included in the computation group. In addition, the blockchain network serves as a mean of storing nodes' identities and computation results, with the aim of ensuring that they cannot be tampered with. It is established among the edge-layer nodes, which are considered to have sufficient storage capacity. Moreover, an incentive mechanism is introduced to enhance the overall performance of data-parallel distributed learning while ensuring security. Specifically, we consider that a round of global iteration of data-parallel distributed learning involves a predefined number of T rounds for blockchain assisted trust management. This facilitates continuous evaluation of nodes' generated models to timely identify the malicious nodes during the model training process before model aggregation. Thus, each round t can be divided into three phases.

- *Nodes attributes registration:* At the beginning of the first round, each node is required to submit its attributes to the attribute registration node due to the heterogeneity of the

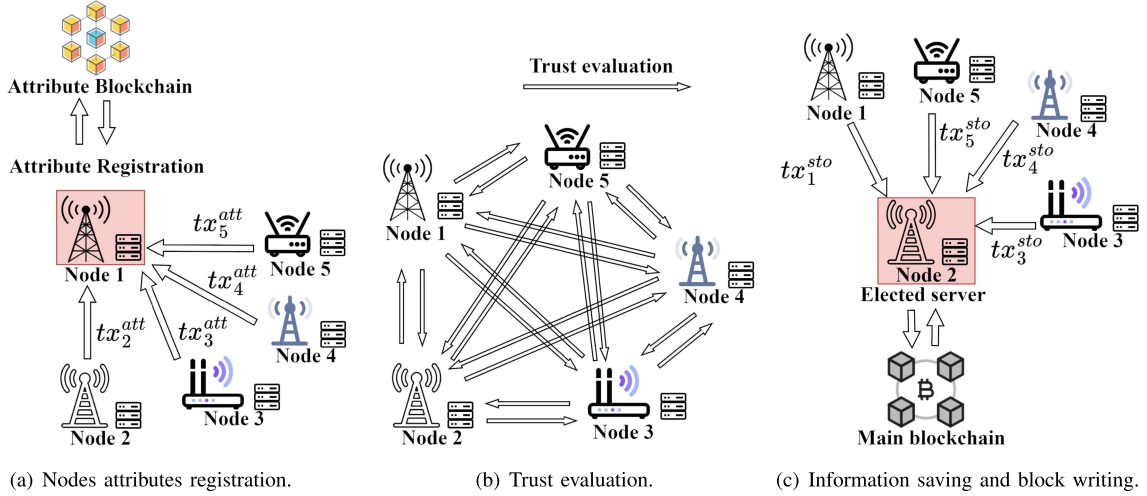


Fig. 2. Illustration of the detailed operations within our proposed scheme.

nodes. The attribute registration node is responsible for verifying the eligibility of nodes' attributes and identities. After that, whenever a new device wants to join the group, it needs to register its attributes to the attribute registration node at the beginning of each round.

- *Trust calculation*: After submitting the attributes to the attribute registration node, each node then conducts local training based on its received training dataset and pre-trained model. The test dataset is also assigned to each node within the computation group to compute evaluations with respect to other nodes, which possess much smaller size than the training dataset. Each node's trust value is then calculated based on the quality value of the local model, evaluations from other nodes, and historical behaviors. Only the nodes with adequate trust value are permitted to remain the computation group. The node with the largest trust value is selected as the server and attribute registration node for the next round.
- *Information saving and block writing*: After calculating the trust value of each node, the server is responsible for recording the participating nodes' information to the blockchain and receiving the corresponding rewards.

Once the pre-defined number of rounds is completed, the local ML models of nodes within the computation group are packed and sent to the server. The server then executes the model-aggregation to generate the updated global model, marking the end of a round of global iteration.

B. Nodes Attributes Registration

To manage who can participate in the computation group, as shown in Fig. 2(a), we consider that each node is required to register its information and attributes to the registration node for information verification. The nodes' attributes can be divided into identifying attributes and non-identifying attributes. For the identifying attributes, a node's intrinsic properties (i.e., device hardware ID [32]) are utilized to uniquely identify nodes. Here,

we consider that such properties are objectively existing and cannot be tampered with. Regarding the non-identifying attributes, they are used to describe the devices, including characteristics such as device type, battery level, memory capacity, and other specifications.

Node i 's attributes can be represented as $\mathcal{A}_i = \{a_i^1, a_i^2, \dots, a_i^{M_i}\}$, where M_i is the number of node i 's attributes. The sets of identifying attributes and non-identifying attributes can be denoted as $\mathcal{A}_i^{\text{iden}}$ and $\mathcal{A}_i^{\text{non-iden}}$, respectively, where $\mathcal{A}_i = \mathcal{A}_i^{\text{iden}} \cup \mathcal{A}_i^{\text{non-iden}}$. Specifically, an attribute can be represented as

$$a_i^j = \langle \text{Category_Name}, \text{value} \rangle, \quad (3)$$

where *Category_Name* and *value* denote the type and value of a_i^j , respectively.

The attribute registration node is elected from the nodes based on trust value. The node with the largest trust value is responsible for registering other nodes' attributes. It is noticed that the attribute registration node and the server is randomly appointed among the nodes as the preapproved nodes in the first round.

Once the registration is completed, the attribute registration node is responsible for generating hash digest DeviceID_i using the nodes' identifying attributes and hash function, which can be represented as

$$\text{DeviceID}_i = H(\mathcal{A}_i^{\text{iden}}), \quad (4)$$

where $H(\cdot)$ is the hash function.

After verification by the nodes within the computation group, the attribute registration node then stores the received attributes and DeviceID_i of nodes in the attribute blockchain network [33], with the goal of ensuring that attribute data cannot be tampered with. Only the nodes within the computation group can be allowed to access the attribute blockchain. The format of data to be stored into the attribute blockchain networks can be represented as

$$tx_i^{\text{att}} = \langle \text{DeviceID}_i, \mathcal{A}_i, \text{TimeStamp}, \text{Sig}_{\text{reg}} \rangle, \quad (5)$$

where Sig_{reg} is the signature signed by the registration node. Here, we consider that DeviceID_i serves as the legal identity

of node i . Thus, nodes' intrinsic properties and attributes, along with the registration node's signature and timestamp, are stored in the attribute blockchain. This ensures a one-to-one mapping between the digital identity and the physical entity.

C. Trust Calculation and Block Writing

In our proposed scheme, we adopt the PoS mechanism, which takes the trust value as the stake. Nodes with larger stakes have a higher probability of being elected as the server to generate the block for storing the nodes' information and adding to the blockchain network.

We consider that a node's trust value is determined by the current quality value of its local model, evaluations from other nodes, and a weighted sum of its historical behaviors [10], which can be denoted as

$$Tru_i^t = \chi \left(\beta_1 Qua_i^t + \sum_{j=1, j \neq i}^J Eva_i^{j,t} - dec_i^t \right) + (1 - \chi) \frac{1}{\sum_{n=1}^{t-1} \gamma^{t-n}} \sum_{n=1}^{t-1} \gamma^{t-n} Tru_i^n, \forall i \in \mathcal{I}, \forall j \in \mathcal{J} \setminus \{i\}, \quad (6)$$

where Tru_i^t is node i 's trust value at the t th round, Qua_i^t is node i 's quality value of locally trained model, J is the number of nodes within the computation group, $Eva_i^{j,t}$ is the evaluation value of node j to i , γ is the discount factor, dec_i^t is the value of the trust value punishment function (the details can be seen in Section IV-D), β_1 is a weighted parameter and χ is a constant. It is noticed that J is initially equal to I at the beginning of the first round, and is updated throughout the process of trust management. Here, we adopt the set $\mathcal{J}$ to represent the nodes remaining in the computation group, which can be represented as $\mathcal{J} = \{1, 2, \dots, J\}$. Only the nodes within the computation group are eligible to issue evaluations to other nodes.

Thus, the iterative process of calculating the trust value can be written as

$$Tru_i^{t+1} = \left(1 - \chi \frac{1 - \gamma}{1 - \gamma^t} \right) Tru_i^t + \chi \left(\beta_1 Qua_i^{t+1} + \sum_{j=1, j \neq i}^J Eva_i^{j,t+1} - dec_i^{t+1} \right) - \chi \frac{\gamma - \gamma^t}{1 - \gamma^t} \left(\beta_1 Qua_i^t + \sum_{j=1, j \neq i}^J Eva_i^{j,t} - dec_i^t \right), \forall i \in \mathcal{I}, \forall j \in \mathcal{J} \setminus \{i\}. \quad (7)$$

Specifically, according to [27], [34], the accuracy¹ of the locally trained ML model can be adopted as a criterion to evaluate the quality of the node's local model. Moreover, in Section VII,

¹Note that the accuracy of the ML model ϵ is defined as $\frac{f(\omega^t) - f(\omega^*)}{f(\omega^0) - f(\omega^*)} \leq \epsilon$, where $f(\cdot)$ represents the loss function of the ML model, ω^t represents the parameter of the ML model at the t th training iteration, ω^* represents the parameter of the optimal ML model. Thus, we consider ϵ as the error before the ML model reaches its convergence condition.

we use multi-layer perception (MLP) and convolutional neural network (CNN) as ML models to train on the MNIST dataset or CIFAR-10 dataset to evaluate the quality of the node's local model. Here, we consider that the distributed datasets for the nodes are IID. Moreover, we consider that newly joined nodes in the computation group will be assigned a pre-trained model, denoted as ω^0 , which will serve as the basis for the local iterations of these nodes. Specifically, referring to the definition in [27], the quality value of node i 's local model can be represented as

$$Qua_i^t = loss(\omega^0) - 2 \frac{-(2-L\psi)\psi\vartheta \left(\left(\sum_{j=1}^{t-1} b_i^t \eta_i^{loc,j} \right) + \eta_i^{loc,t} \right)}{2}, \quad \forall i \in \mathcal{I}, \quad (8)$$

where $loss(\omega^0)$ represents the value of the loss function of the pre-trained model ω^0 , and the second term of (8) represents the loss function of the current model ω_i^t . L , ψ and ϑ are the parameters related to node i 's local iteration. Here, we limit the value range of Qua_i^t as $[-1, 1]$. It is worth pointing out that $\{b_i^t \in \{0, 1\}\}_{\forall i \in \mathcal{I}}$ is a set of binary variables denoting the nodes-scheduling at the t th round, where $b_i^t = 1$ denotes that node i is permitted to participate in the computation group at the t th round, and $b_i^t = 0$ otherwise.

After the nodes' local iterations are finished, as shown in Fig. 2(b), they need to transmit the results of their local iterations to other nodes for evaluation. The format of the message for nodes to transmit is represented as

$$tx_i^{loc} = \langle Qua_i^t, \omega_i^t, \text{TimeStamp}, Sig_i \rangle, \quad (9)$$

where Sig_i is the signature signed by node i and ω_i^t represents node i 's locally trained ML models at the t th round.

As for the evaluation of node j to node i , we consider that node i needs to transmit its locally trained ML model to node j for evaluation. Node j is required to use its received test dataset to evaluate the performance of node i 's ML model and calculate the corresponding quality value of the local model. It is noticed that the training and test datasets of all nodes are IID, which implies that a model trained on one node's dataset can achieve the similar performance on the datasets of other nodes. Thus, the evaluation of node j to node i can be represented as

$$Eva_i^{j,t} = \beta_2 Rea_i^{j,t} Qua_i^{j,t}, \forall i \in \mathcal{I}, \forall j \in \mathcal{J} \setminus \{i\}, \quad (10)$$

where $Rea_i^{j,t}$ is the interactive weight between node i and node j at the t th round, $Qua_i^{j,t}$ is the evaluated quality value of node i 's local model measured by node j , and β_2 is the weighting parameter. It is noteworthy that the value of $Rea_i^{j,t}$ is related to whether node i and node j participate in the computation group or not. The initial value of $Rea_i^{j,t}$ is set as ν_0 . If node i and node j participate in the computation group simultaneously, the value of $Rea_i^{j,t}$ will be increased by $\Delta\nu$. Otherwise, the value of $Rea_i^{j,t}$ will be reduced by $\Delta\nu$.

After the process of evaluation, the result of node j 's evaluation for node i is required to be broadcasted within the computation group. The format of the message can be represented as

$$tx_i^{eva,j} = \langle Eva_i^{j,t}, Qua_i^{j,t}, \text{TimeStamp}, Sig_j \rangle, \quad (11)$$

where Sig_j represents the signature of node j .

We consider the size of node i 's test dataset as D_i^{test} . Thus, the energy consumption for node i to calculate j 's evaluation value can be represented as

$$E_i^{\text{cal},j,t} = \kappa C D_i^{\text{test}} \mu_i^2, \forall j \in \mathcal{I}, \forall i \in \mathcal{J} \setminus \{j\}. \quad (12)$$

Similarly, the latency for node i to calculate node j 's evaluation value can be represented as

$$l_i^{\text{cal},j,t} = \frac{C D_i^{\text{test}}}{\mu_i}, \forall j \in \mathcal{I}, \forall i \in \mathcal{J} \setminus \{j\}. \quad (13)$$

Based on (12) and (13), the total energy consumption and latency of node i to compute the evaluation of other nodes can be represented as

$$E_i^{\text{cal},t} = \sum_{j=1}^I E_i^{\text{cal},j,t}, \forall j \in \mathcal{I}, \forall i \in \mathcal{J} \setminus \{j\}, \quad (14)$$

$$l_i^{\text{cal},t} = \sum_{j=1}^I l_i^{\text{cal},j,t}, \forall j \in \mathcal{I}, \forall i \in \mathcal{J} \setminus \{j\}. \quad (15)$$

Therefore, the energy consumption of node i can be expressed as

$$E_i^t = E_i^{\text{loc},t} + E_i^{\text{cal},t}, \forall i \in \mathcal{I}, \quad (16)$$

where $E_i^{\text{loc},t}$ and $E_i^{\text{cal},t}$ denote the energy consumption for local computation and trust calculation, respectively.

Besides, the latency of node i can be represented as

$$l_i^t = l_i^{\text{loc},t} + l_i^{\text{cal},t}, \forall i \in \mathcal{I}, \quad (17)$$

where $l_i^{\text{loc},t}$ and $l_i^{\text{cal},t}$ denote the latency for local computation and trust calculation, respectively.

To effectively filter out the nodes that transmit the abnormal value of the local model's quality value or evaluation, we introduce ρ to denote the tolerable bias for the quality value of node i 's local model and other nodes' evaluated quality values for node i . Moreover, according to [28], the euclidean distances between models can be utilized to distinguish benign models and malicious models that may engage in poisoning attacks, which is represented as $Dis(\omega_i, \omega_j) = \sqrt{\sum (\omega_i - \omega_j)^2}$. The differences in euclidean distances between benign and poisoned local models are much larger than those observed between benign local models. Based on this feature, the MaxClique method [35] provides an approach to identify benign models according to the euclidean distances between models. Thus, we propose an algorithm combining the principle of the MaxClique method and the nodes' quality values to filter out the malicious nodes. The details are shown in Algorithm 1.

From Step 4 to Step 11, we calculate euclidean distances between each node's local model to construct neighbor sets. Each neighbor set is then filtered using a tolerable bias ς . From Step 12 to Step 20, the average value of the quality value and evaluated quality value of the node's local model is then used to filter the nodes with abnormal quality values or evaluation values. As we consider that the proportion of the malicious nodes among all nodes does not exceed 1/3 [36], if the size of the

Algorithm 1: Filtering Process to Identify Benign Nodes.

```

1: Initialization: Initialize the tolerable bias  $\rho$ ,  $\varsigma$  for the
   euclidean distances and nodes' quality values,
   respectively. Let  $\mathcal{W}^{\text{benign}}$  represents the set of benign
   nodes' local models. Let  $\Delta\varsigma$  be a very small positive
   number.
2: Input: The set of the nodes' locally trained models
    $\{\omega_i^t\}_{\forall i \in \mathcal{I}}$ , the nodes' quality values  $\{Qua_i^t\}_{\forall i \in \mathcal{I}}$  and
   evaluated quality value  $\{Qua_i^{j,t}\}_{\forall i,j \in \mathcal{I}}$ .
3: while  $|\mathcal{W}^{\text{benign}}| \neq 0$  do
4:   for  $i$  in range( $I - 1$ ) do
5:     Set the neighbor set of node  $i$ 's model  $\mathcal{N}_i$  as the
     empty set.
6:     for  $j$  in range( $i + 1, I$ ) do
7:       if  $Dis(\omega_i, \omega_j) \leq \varsigma$  then
8:          $\mathcal{N}_i = \mathcal{N}_i \cup \omega_j$ .
9:       end if
10:    end for
11:  end for
12:  for each neighbor set  $\mathcal{N}_i$  do
13:    for each model  $k$  in  $\mathcal{N}_i$  do
14:      if  $|Qua_k^t - \frac{1}{|\mathcal{N}_i|}(Qua_k^t + \sum_{j \in \mathcal{N}_i, j \neq k} Qua_k^{j,t})| > \rho$ 
        then
15:         $\mathcal{N}_i = \mathcal{N}_i \setminus \omega_k^t$ .
16:      else if
         $|Qua_k^{j,t} - \frac{1}{|\mathcal{N}_i|}(Qua_k^t + \sum_{j \in \mathcal{N}_i, j \neq k} Qua_k^{j,t})| > \rho$ 
        then
17:         $\mathcal{N}_i = \mathcal{N}_i \setminus \omega_j^t$ .
18:      end if
19:    end for
20:  end for
21:  Find the largest neighbor set  $\mathcal{N}_i^{\text{max}}$ .
22:  if  $|\mathcal{N}_i^{\text{max}}| \geq \frac{2I}{3}$  then
23:     $\mathcal{W}^{\text{benign}} = \mathcal{N}_i^{\text{max}}$ .
24:  end if
25:   $\varsigma = \varsigma + \Delta\varsigma$ .
26: end while
27: Output: The set of benign nodes' local models  $\mathcal{W}^{\text{benign}}$ .

```

largest neighbor set exceeds $2I/3$, the local models within the set and corresponding nodes are considered benign. Otherwise, the tolerable bias ς increases with a small step-size $\Delta\varsigma$.

In addition, we use δ^t to denote the threshold of the trust value at the t th round. We consider that only nodes with trust value greater than δ^t are allowed to remain in the computation group, which is represented as

$$Tru_i^t \geq \delta^t, \forall i \in \mathcal{I}. \quad (18)$$

In particular, δ^t can be used to avoid nodes with insufficient trust value from participating in the computing group. Thus, to ensure the computing quality of the computation group, we can represent the constraint of δ^t as

$$\delta^t \geq \frac{1}{I} \sum_{i=1}^I Tru_i^t, \forall i \in \mathcal{I}. \quad (19)$$

After each node's trust value is calculated, the node with the highest trust value is elected to be the server, which is responsible for nodes' attributes registration and data storage.

After the nodes which can remain in the computation group are determined, as shown in Fig. 2(c), the server is responsible for storing the nodes' data, including timestamps, local models, trust values, and signatures, in the main blockchain to avoid leakage of sensitive information or data manipulation. The format of data for storing in the blockchain can be represented as

$$tx_i^{\text{sto}} = \langle \omega_i^t, Tru_i^t, \text{TimeStamp}, Sig_i \rangle. \quad (20)$$

The server is responsible for collecting all nodes' data and packing them into the body of the block. It is noteworthy that the subsequent iteration round of local model training for the member nodes of the computation group to generate ω_i^{t+1} will be based on the model ω_i^t they stored in the block in the current round.

Before appending the block to the blockchain, the body of the block is required to be verified by the nodes within the computation group. The server broadcasts the block to each node. All nodes within the computation group verify the received block. After the verification process, each node transmits the confirmation message with their signatures back to the server. If the server receives more than two-thirds of the nodes' (i.e., $\geq \frac{2}{3}J + 1$) confirmation messages within the specified time limit, it will append the block to the blockchain.

Moreover, we consider that our proposed scheme includes on-chain and off-chain components. The computation-intensive and data-intensive tasks, such as model training, trust calculation, and model evaluation, are processed off-chain. The on-chain components are responsible for information saving and local models aggregation. This design facilitates a large number of participating nodes and improves overall efficiency, overcoming the performance limitations of the blockchain [31].

D. Incentive Mechanism for Nodes Behaviors

To be eligible to participate in the computation group, each node needs to improve its trust value, which may cause extra energy consumption, latency, and potential security threats. Besides, some malicious nodes may conduct malicious behaviors via deliberately downgrading the evaluation value for other nodes or disseminating spurious trust values.

To motivate nodes to obey the proposed scheme and improve their trust value to participate in the computation group, we consider that the nodes with positive behaviors can be given corresponding awards, which can be represented as

$$\text{Award}_i^t = P^t + M^t \text{Qua}_i^t, \forall i \in \mathcal{I}, \quad (21)$$

where Award_i^t denotes node i 's reward at the t th round, $M^t \text{Qua}_i^t$ denotes the rewards obtained by participating in the computation group and P^t denotes the reward for the server node who executes blockchain storage and block generation successfully, which can be represented as

$$P^t = \begin{cases} r, & \text{node } i \text{ is selected as the server,} \\ 0, & \text{Otherwise.} \end{cases} \quad (22)$$

Here, we consider that the nodes' rewards obtained by participating in the computation group are related to their local models' quality values. Larger reward value is given to those nodes with higher quality values, which thus motivates the nodes to increase the number of their local iterations and obtain the higher quality values.

As for the nodes that engage in malicious behaviors, we also take punitive measures against them. Different types of malicious behaviors have different effects in terms of the reliability and security of the system. Thus, the trust value punishment function for different types of malicious behaviors can be represented as

$$\text{dec}_i^t = \sum_{t_i^{\text{mal}} \in \mathcal{T}_i^{\text{mal}}} \frac{t_i^{\text{mal}} + a}{t + a} v_{\text{dec}} Tru_i^{t_i^{\text{mal}}}, \forall i \in \mathcal{I}, \quad (23)$$

where node i conducts the malicious behavior at the t_i^{mal} th round, a represents the decay factor, and v_{dec} is the trust value punishment function for different types of malicious behaviors, which can be defined as

$$v_{\text{dec}} = \begin{cases} v_{\text{dec}_1}, & \text{invasion,} \\ v_{\text{dec}_2}, & \text{fake identity,} \\ v_{\text{dec}_3}, & \text{fake data.} \end{cases} \quad (24)$$

Among the threat models mentioned above, invasion comprises data poisoning attacks and historical behaviors tampering attacks. Misbehavior of fake identity includes sybil and newcomer attacks, while misbehavior of fake data encompasses self-promoting, bad-mouthing and ballot-stuffing attacks. Moreover, we consider $|v_{\text{dec}_1}| > |v_{\text{dec}_2}| > |v_{\text{dec}_3}|$, which reflects different levels of severity of malicious behavior.

Similarly, the value of the punishment for node's utility can be represented as

$$\text{Punish}_i^t = \begin{cases} v_{\text{pun}_1}, & \text{invasion,} \\ v_{\text{pun}_2}, & \text{fake identity,} \\ v_{\text{pun}_3}, & \text{fake data.} \end{cases} \quad (25)$$

Similar to the settings of the trust value punishment function, we consider $|v_{\text{pun}_1}| > |v_{\text{pun}_2}| > |v_{\text{pun}_3}|$. It is worth mentioning that the value of punishment for node's utility is much higher than the value of reward for node's utility, which motivates nodes to avoid malicious behaviors to gain higher revenue.

The trust punishment v_{dec} and utility punishment Punish_i^t for different types of malicious behaviors are predefined in the proposed scheme. The methods for identifying malicious behavior and the corresponding mitigation strategies will be discussed in Section VII-D.

The nodes who are permitted to join the computation group will consume energy and incur latency [27]. Thus, the cost for a node to join the calculation system can be expressed as

$$C_i^t = \alpha_1 E_i^t + \alpha_2 L_i^t, \forall i \in \mathcal{I}, \quad (26)$$

where E_i^t and L_i^t denote the energy consumption and the latency of node i at the t th round, respectively. Parameters α_1 and α_2 denote the weighted factors.

Based on the cost, award and punishment, the expected utility of node i can be expressed as

$$U_i^t = Award_i^t - \alpha_3 C_i^t - \alpha_4 Punish_i^t, \forall i \in \mathcal{I}, \quad (27)$$

where α_3 and α_4 are the weighted factors.

As mentioned in [37], the rewards can be instantiated as a specific type of currency within the blockchain and the node's system cost can be considered as their expended reward. Therefore, the node's overall utility represents the ultimate benefit they receive during the process of distributed learning.

V. OPTIMIZATION PROBLEM FORMULATION

Based on the discussions above, we formulate a joint optimization problem that optimizes the number of node i 's local iterations $\eta_i^{\text{loc},t}$ and the threshold of node's trust value δ^t , with the goal of maximizing the overall utility of all nodes within a round of global iteration.

$$\max \sum_{i=1}^I \sum_{t=1}^T U_i^t$$

$$\text{subject to: } Tru_i^t \geq \delta^t, \forall i \in \mathcal{I}, \quad (28)$$

$$\delta^t \geq \frac{1}{I} \sum_{i=1}^I Tru_i^t, \forall i \in \mathcal{I}, \quad (29)$$

$$0 \leq \eta_i^{\text{loc},t} \leq \eta_i^{\text{loc},\max}, \forall i \in \mathcal{I}, \quad (30)$$

$$0 \leq E_i^{\text{loc},t} + E_i^{\text{cal},t} \leq E_i^{\max}, \forall i \in \mathcal{I}, \quad (31)$$

$$0 \leq l_i^{\text{loc},t} + l_i^{\text{cal},t} \leq l_i^{\max}, \forall i \in \mathcal{I},$$

$$\text{variables: } \eta_i^{\text{loc},t} \text{ and } \delta^t. \quad (32)$$

Constraint (28) ensures that only nodes with trust values exceeding the threshold are allowed to participate in the computation group. Constraint (29) ensures that the threshold of the trust value should be higher than its lower-bound. Constraint (30) ensures that the number of local iterations for each node remains within the upper limit, denoted by $\eta_i^{\text{loc},\max}$. Constraints (31) and (32) ensure that the node's energy consumption and latency cannot exceed the upper-bounds denoted by $E_i^{\max}$ and $l_i^{\max}$, respectively. Besides, to enable more flexible adjustment and optimization, we consider the threshold of the trust value as a continuous variable.

Our formulated problem can be defined as a dynamic programming problem over the finite time-horizons. Nevertheless, the number of participating nodes may change dynamically during the period of the iteration process and the threshold of the trust value is a continuous variable, which makes it hard to adopt the static optimization to solve the proposed problem. To address this difficulty and effectively solve the proposed problem, we model the proposed problem as an MDP problem and design an optimization algorithm based on DRL.

VI. PROPOSED OPTIMIZATION ALGORITHM

In MDP, the state space, action space, rewards, and the state transition can be represented as follows.

- *State Space*: We use $State(t)$ to represent the state space at the t th round of iteration, which can be expressed as

$$State(t) = \left\{ \{E_i^t\}_{i \in \mathcal{I}}, \{l_i^t\}_{i \in \mathcal{I}}, \{Tru_i^t\}_{i \in \mathcal{I}}, \{Rea_i^{j,t}\}_{i,j \in \mathcal{I}}, \sum_{i=1}^I \sum_{k=1}^t U_i^k \right\}, \quad (33)$$

which includes each node's energy consumption E_i^t , latency l_i^t , trust value Tru_i^t , interactive weights $Rea_i^{j,t}$ and the cumulative value of the utility function U_i^t .

- *Action Space*: We use $Action(t)$ to represent the action that includes the number of node's local iterations $\eta_i^{\text{loc},t}$ and the threshold of node's trust value δ^t . Thus, we can express the action space $Action(t)$ as

$$Action(t) = \left\{ \{\eta_i^{\text{loc},t}\}_{i \in \mathcal{I}}, \delta^t \right\}. \quad (34)$$

- *Reward*: We use node's utility function value to represent the value of the reward after executing the determined actions, i.e.,

$$Reward(t) = \sum_{i=1}^I U_i^t - \zeta \tau(State(t), Action(t)), \quad (35)$$

where ζ represents the weighted parameter of the penalty function, and $\tau(State(t), Action(t))$ represents the penalty function, which can be expressed as

$$\begin{aligned} \tau(State(t), Action(t)) &= \begin{cases} 0, & (30), (31), \text{ and } (32) \text{ are met,} \\ 1, & \text{otherwise.} \end{cases} \end{aligned} \quad (36)$$

It should be noted that the reward function in (35) is not strictly equivalent to the objective function of the proposed optimization problem. The inclusion of the penalty function discourages the agent from exploring actions that are considered invalid, thus preventing potential distortions in policy learning.

- *State Transition*: After determining and executing the actions of nodes within each iteration round, we obtain the reward value and the next state, which can be expressed as

$$State(t+1) = \left\{ \{E_i^{t+1}\}_{i \in \mathcal{I}}, \{l_i^{t+1}\}_{i \in \mathcal{I}}, \{Tru_i^{t+1}\}_{i \in \mathcal{I}}, \{Rea_i^{j,t+1}\}_{i,j \in \mathcal{I}}, \sum_{i=1}^I \sum_{k=1}^{t+1} U_i^k \right\}. \quad (37)$$

Specifically, the transition of the node's trust value from Tru_i^t to Tru_i^{t+1} is illustrated by (7) before. Besides, the transition of connection weight between $Rea_i^{j,t}$ and $Rea_i^{j,t+1}$ can be represented as

$$Rea_i^{j,t+1} = \begin{cases} Rea_i^{j,t} + \Delta\nu, & \text{both node } i \text{ and node } j \\ & \text{participate at the} \\ & t\text{-th iteration round,} \\ Rea_i^{j,t} - \Delta\nu, & \text{otherwise.} \end{cases} \quad (38)$$

Until now, we have transformed the original problem into an MDP problem. To solve the proposed MDP problem, the optimal policy for scheduling each node's number of local iterations and the threshold of the trust value needs to be learned by observing the state transitions and rewards generation under the given policy, with the goal of achieving the maximum utility of nodes within a specified time period.

To tackle this issue, we adopt the actor-critic (AC) [38] network based DRL algorithm to solve it. As a deep reinforcement learning algorithm, the AC network can be adopted to effectively solve the MDP problem with multi-dimensional and continuous space. We use θ to denote the vector of the actor-network and ϕ to denote the vector of the critic-network. Thus, the AC network based DRL algorithm can be utilized to obtain the optimal policy $\pi_{\theta}(Action(t)|State(t))$ that yields the $Action(t)$ under the $State(t)$. To prevent unstable updates of the actor-network and the critic-network parameters, we utilize PPO with importance sampling weights to update these parameters.

Specifically, we consider the framework of the DRL algorithm contains three parts: environment entity, offline learning, and online decision. In the offline learning part, the DRL agent utilizes theoretical values to determine state transitions and their corresponding rewards, which are then used for policy learning. As for the online decision part, the instant decisions provided by PPO are utilized to act as the optimized real-time decision-making to select the optimal nodes' local iterations and the threshold of the trust value under the real data-sets. The details of the environment entity, the actor, and the critic neural networks and their update policies are illustrated as follows.

- *Environment entity*: The environment entity of the DRL algorithm's framework is utilized to illustrate the constraints among the number of node i 's local iterations $\eta_i^{loc,t}$ and the threshold of the trust value δ^t in the proposed problem. Besides, the environment entity is also used to describe the blockchain assisted trust management scheme via the PoS mechanism which generates $State(t+1)$ and $Reward(t)$ under the given $State(t)$ and $Action(t)$. Thus, the policy $\pi_{\theta}(Action(t)|State(t))$ generated by the DRL algorithm can be executed within the environment entity to obtain the corresponding reward and updated state. It is worth noticing that during the offline learning part, (7), (8), (16), and (17) are utilized to generate the node's quality value, the node's system cost, and the corresponding state transitions of the environment entity. Besides, the measured quality values of node j to node i are set equal to node i 's calculated quality value during the offline learning. As for the online decision part, real data-sets MNIST and CIFAR-10 are utilized as the data in the real online environment to calculate the node's quality value and trust value.
- *Actor neural network*: The role of the actor neural network is to generate the optimal policy for solving the proposed MDP problem. Here, we use actor neural network to generate the mean and variance of the Gaussian normal distribution [39]. Specifically, the mean and variance of the Gaussian normal distribution can be represented as $\mu_{Action(t)} = (\mu_{\eta_1^t}, \mu_{\eta_2^t}, \dots, \mu_{\eta_l^t}, \mu_{\delta^t})$ and $\sigma_{Action(t)}^2 =$

$(\sigma_{\eta_1^t}^2, \sigma_{\eta_2^t}^2, \dots, \sigma_{\eta_l^t}^2, \sigma_{\delta^t}^2)$, respectively. Based on the mean and variance of the Gaussian normal distribution, we can express the probability density function of policy for action scheduling as

$$\pi_{\theta}(Action(t)|State(t)) = \frac{1}{\sqrt{2\pi}\sigma_{Action(t)}} \exp\left(\frac{-(Action(t) - \mu_{Action(t)})^2}{2\sigma_{Action(t)}^2}\right). \quad (39)$$

Here, we use the clipped surrogate objective to obtain the gradient, which can be written as

$$\nabla_{\theta} J(\theta) = \min \left[\frac{\pi_{\theta}}{\pi_{\theta_{old}}} Adv_{\pi_{\theta_{old}}}(State, Action), \right. \\ \left. clip\left(\frac{\pi_{\theta}}{\pi_{\theta_{old}}}, 1 - \varepsilon, 1 + \varepsilon\right) Adv_{\pi_{\theta_{old}}}(State, Action) \right], \quad (40)$$

where ε is a factor utilized in PPO that limits the value range of $\nabla_{\theta} J(\theta)$, with the goal of limiting the policy update within the interval of $[1 - \varepsilon, 1 + \varepsilon]$ and preventing large policy updates that can lead to instability.

We consider that $Adv_{\pi_{\theta}}(State, Action)$ denotes the advantage function that captures the temporal differential errors to update the vector of the actor-network. The form of the advantage function can be written as

$$Adv_{\pi_{\theta}}(State, Action) = \hat{R}^{\pi}(State, Action) - V_{\phi}^{\pi}(State(t)). \quad (41)$$

We can update the vector of the actor neural network as

$$\theta = \theta_{old} + \varrho_A \nabla_{\theta} J(\theta). \quad (42)$$

- *Critic neural network*: The critic neural network plays a crucial role in evaluating the effectiveness of the policy generated by the actor neural network. Similar to the actor neural network, we also use the temporal differential errors to calculate the gradient of the loss function $\nabla_{\phi} L(\phi)$ to update the critic neural network, which can be written as

$$\nabla_{\phi} L(\phi) = Reward(t) + \gamma * (V_{\phi}^{\pi}(State(t+1)) - V_{\phi}^{\pi}(State(t))), \quad (43)$$

where we use $V_{\phi}^{\pi}(State(t))$ to denote the expected reward under the current policy π .

Thus, we can update the vector of the critic neural network as

$$\phi = \phi_{old} + \varrho_C \nabla_{\phi} L(\phi). \quad (44)$$

The details of PPO based algorithm to solve our proposed problem are illustrated in Algorithm 2. The three-tuple $(State(t+1), Action(t), Reward(t))$ is stored in the replay memory buffer, serving as a record of exploration. Algorithm 3 is invoked to update the vectors of the actor neural network and critic neural network. It is noticed that the malicious behaviors of

Algorithm 2: PPO Based Algorithm to Solve Our Proposed Problem.

```

1: Initialization: Initialize the vector of actor-network  $\theta$ ,
   the vector of critic-network  $\phi$ , and the replay memory
   buffer. Set the number of episodes as  $T$  and  $K$  and the
   size of the replay memory buffer as  $L_{\text{Buffer}}$ .
2: for  $k = 1, 2, \dots, K$  do
3:   Initialize the environment entity and initial state
    $State(1)$ .
4:   for  $t = 1, 2, \dots, T$  do
5:     Execute current policy  $\pi_{\theta}(Action(t)|State(t))$ 
     using the actor-network under state  $State(t)$  to
     derive action  $Action(t)$ .
6:     Execute derived action  $Action(t)$  in the
     environment entity to derive state  $State(t+1)$  and
     corresponding reward  $Reward(t)$ .
7:     Store three-tuple
      $(State(t+1), Action(t), Reward(t))$  in the replay
     memory buffer.
8:   end for
9:   if  $k \% L_{\text{Buffer}} == 0$  then
10:    Sample the data from the replay memory buffer.
11:    Invoke Algorithm 3 to update the actor-network and
    the critic-network and derive the updated vector of
    actor-network  $\theta$  and the updated vector of
    critic-network  $\phi$  based on the sampled data.
12:    Initialize the replay memory buffer.
13:   end if
14: end for
15: Output: Policy  $\pi_{\theta}$  using the optimized actor-network
    for scheduling nodes' local iterations, the threshold of
    the trust values and total reward  $\sum_{i=1}^I \sum_{t=1}^T U_i^t$ .

```

nodes are not taken into account during the offline learning of our proposed DRL algorithm. The goal of the proposed algorithm is to output the optimal policy for the edge-layer nodes to achieve reliable and cost-efficient model convergence. Due to the mechanism for delaying rewards, our proposed algorithm can optimize the long-term performance under dynamically changing system states with each decision, which outperforms the myopic heuristic policy.

VII. PERFORMANCE EVALUATION AND SECURITY ANALYSIS

In this section, we provide the experimental results to show the performance and the advantage of our proposed algorithm. We propose a scenario in which a cluster of nodes is distributed within a circular region. The parameter-settings used in the simulation are presented in Table I. We first implement our proposed PPO based algorithm via Python 3.7 to generate the optimal policy and verify its convergence and performance under different parameter settings. We then evaluate the optimal policy under the PoS enabled blockchain network.

Algorithm 3: Clipped Surrogate Objective Based PPO to Train and Update the Actor and the Critic Neural Network.

```

1: Initialization: Set the updating steps of actor neural
   network and critic neural network  $\varphi_A$  and  $\varphi_C$ , and
   discounted parameter  $\gamma$ , and clipped parameter  $\epsilon$ .
Input: The state of the last state space  $State(t)$ , and the
   vector of actor neural network  $\theta$ , and the vector of critic
   neural network  $\phi$ .
2: Set  $\hat{R}^{\pi}(t) = Reward(t) + \gamma * V^{\pi}(State(t))$ .
3: for  $k = t - 1, t - 2, \dots, 1$  do
4:   Calculate the discounted reward
    $\hat{R}^{\pi}(k) = Reward(k) + \gamma * \hat{R}^{\pi}(k + 1)$ .
5: end for
6: Calculate the Advantage function using the discounted
   reward and state value and (41).
7: for  $i = 1, 2, \dots, \varphi_A$  do
8:   Calculate the gradient of actor deep neural network
    $\nabla_{\theta} J(\theta)$  using (40).
9:   Update the vector of actor neural network  $\theta$  using (42).
10: end for
11: for  $i = 1, 2, \dots, \varphi_C$  do
12:   Calculate the gradient of critic deep neural network
    $\nabla_{\phi} L(\phi)$  using (43).
13:   Update the vector of critic neural network  $\phi$ 
   using (44).
14: end for
15: Output: The updated vector of actor neural network  $\theta$ ,
   and the updated vector of critic neural network  $\phi$ .

```

TABLE I
PARAMETER SETTINGS

Parameters	Settings
The CPU-cycles/bit, C	1000
The CPU power-efficiency, κ	10^{-28}
The upper-bound of the number of node's local iteration, $\eta_{i, \text{loc}, t, \text{max}}$	20
The total number of nodes, I	[4, 6, 8]
The node's CPU's processing capability, μ_i	1GHz
The step-size for updating actor network, ϱ_A	$1.2 * 10^{-4}$
The step-size for updating critic network, ϱ_C	$1.2 * 10^{-4}$
The number of iteration rounds of DRL algorithm, K	$5 * 10^5$
The number of hidden layer's neuron	4096

A. Convergence of Our Proposed Algorithm

Fig. 3 shows the convergence of the values of the utility function achieved by our proposed PPO based algorithm under the different number of nodes. Here, we set the clip parameter of the PPO based algorithm as 0.2 and the number of iteration episodes as 1000. The number of updating steps for actor network φ_A and critic network φ_C are set as 10. We set the size of the dataset allocated to each node as 8 Mbits and the number of overall rounds as 5. We also vary the number of nodes from 6 to 10. During each iteration episode, the number of nodes' local iterations and the threshold of the trust value are determined via the PPO based algorithm. As shown in Fig. 3, we can observe that our proposed PPO based algorithm can always converge

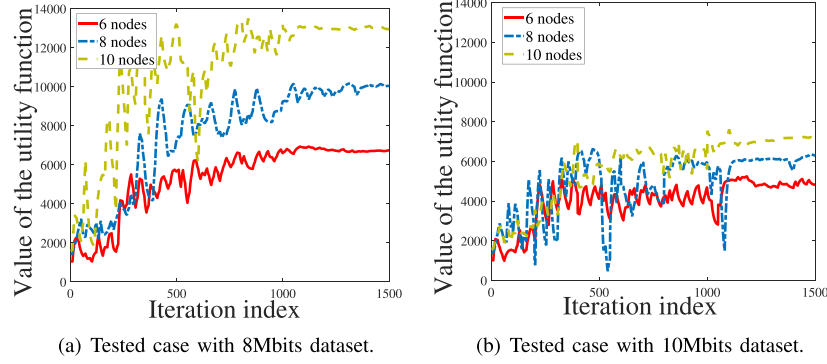


Fig. 3. Illustration of the convergence of proposed PPO based Algorithm.

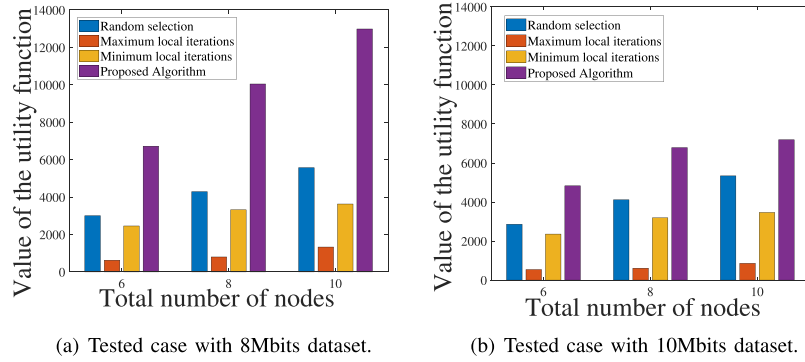


Fig. 4. Advantages of our proposed PPO based algorithm against the fixed local iterations.

under different number of nodes. Besides, we can also observe from Fig. 3 that the larger number of nodes results in a larger value of the utility function. This is because the optimized policy can always obtain awards greater than the costs, leading to an overall increase in utility as the number of nodes increases.

B. Performance of Our Proposed Algorithm Under Different Settings

Fig. 4 shows the advantage of our proposed PPO based algorithm in comparison with the following baseline schemes.

- *Minimum local iterations*: This scheme employs the minimum number of nodes' local iterations to save the corresponding cost.
- *Maximum local iterations*: This scheme proposed in PoSBFL [40] employs the maximum number of nodes' local iterations to make the most learning contributions.
- *Random selection*: This scheme employs the randomly selected nodes' local iterations within the interval $[1, \eta_i^{\text{loc}, \text{max}}]$.

These cases correspond to different levels of accuracy for the locally trained ML models and quality values. We vary the total number of nodes to 6, 8, and 10, and allocate dataset sizes of 8 and 10 Mbits to each node. The number of T rounds within one cycle is set to 5. The results in Fig. 4 verify that our proposed PPO based algorithm can obtain the larger value of the utility function, which thus outperforms the heuristic schemes. Besides, the

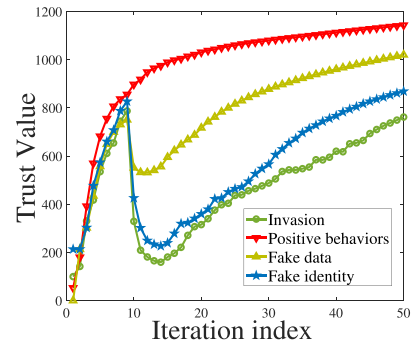


Fig. 5. Evolution of the nodes' trust values with various kinds of behaviors.

results in Fig. 4 also validate the importance of properly setting the number of local iterations to generate high enough quality value to participate in the computation group, while concurrently preventing excessive latency and energy consumption.

Fig. 5 shows the evolution of the nodes' trust values with various kinds of behaviors. Here, we consider that the legitimate nodes always conduct positive behaviors during the iterative process. As for the malicious nodes, they may conduct malicious behaviors, including invading the system, forging identities, and providing false data during certain rounds of the iterative process. We can observe from Fig. 5 that the trust values of the nodes with positive behaviors keep growing throughout the iterative process. Regarding the nodes that engage in malicious

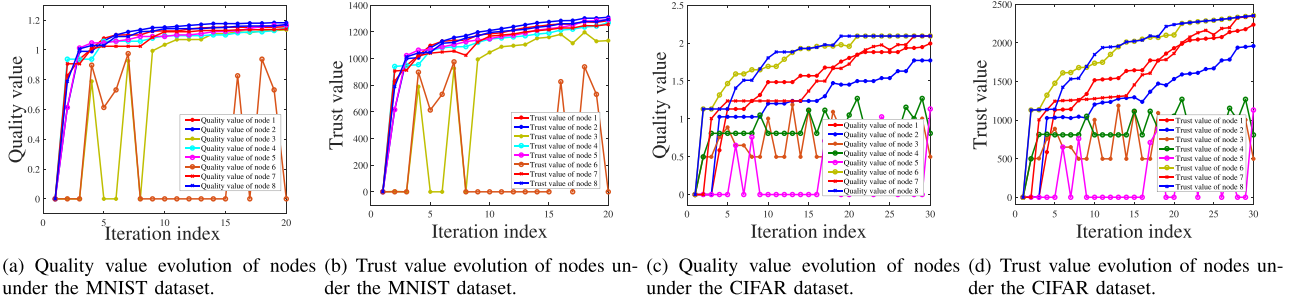


Fig. 6. Evolution of the trust value and quality value of nodes during the process of iteration under MNIST and CIFAR dataset.

behaviors, we can see that the malicious behaviors can dramatically decrease the value of nodes' trust values once they are identified and detected. This is because the malicious behaviors are penalized and result in time decay of trust values. Thus, the malicious nodes may suffer a significant drop in trust values and cannot recover quickly even if they adopt positive behaviors in the following iterative process. Furthermore, the results in Fig. 5 demonstrate that nodes are incentivized to act in an honest manner in order to avoid being penalized and experiencing a decrease in their trust values.

C. Performance of Our Proposed Scheme Under PoS Enabled Blockchain Network

We implement a PoS enabled blockchain network to simulate a real-world environment in this subsection. We implement the blockchain network via GO language and simulate eight nodes within the network. We first run the proposed AC-PPO algorithm to obtain the optimal policy for nodes' local iterations and selection in the edge layer. The obtained optimal policy is then used for the data-parallel distributed learning, while the blockchain network is also employed to execute the consensus algorithm and manage data storage. The edge-layer nodes are interconnected and can compete to become server based on their respective trust values. As for the nodes' computation tasks, we adopt MLP and CNN as the ML models for the nodes to conduct local computation-tasks and generate locally trained ML models, with the objective of executing the application of the image classification on MNIST dataset or CIFAR-10 dataset. The candidate block is generated by the server, which consists of the parameter of the locally trained ML model, trust value, and node's signature. The candidate block is then broadcasted to other nodes for verification. Once validated, the block is added to the blockchain.

Fig. 6 illustrates the evolution of nodes' trust and quality values in our proposed scheme, using the MNIST and CIFAR datasets with MLP and CNN models. We set the ML model's learning rate to 0.01, batch size to 100, and the number of nodes to 8. In Fig. 6(a) and (b), we train MLP models on the MNIST dataset, with each node assigned 10 Mbits of data using IID sampling. We conduct 20 training rounds, limiting each node to a maximum of 20 local iterations per round. In Fig. 6(c) and (d), we further train CNN models on the CIFAR dataset with each user assigned 70 Mbits of data using an IID sampling strategy.

We conduct 30 training rounds, with a maximum of 130 local iterations per node in each round. From 6(a) and (c), we observe that the quality values of nodes start from zero at the beginning of the iteration and reach the maximum boundary at about 5 to 20th iteration round and remain steady thereafter. For nodes that are unable to participate in the computation group, their quality values remain fixed throughout the evolution process. This is because their locally trained ML models cannot be stored in the blockchain, forcing their local iteration process to start from scratch each time. As shown in Fig. 6(b) and (d), the trust values of nodes in the computation group experience rapid growth during the first 5 to 20 iteration rounds, and then the growth trend becomes flat. Although their trust values eventually converge to similar maximum boundaries, each node's convergence rate differs, indicating different training policies during the process of iteration. As for the nodes that cannot participate in the computation group, their trust values keep fluctuating in the iteration process. This indicates that these nodes have made unsuccessful attempts to meet the required threshold and join the computation group. Overall, the results in Fig. 6 show the effectiveness of our proposed scheme in preventing nodes with insufficient trust values from participating in the computation group and resisting low-quality data dissemination, thereby motivating nodes to act honestly and improve their trust values.

In Fig. 7, we use MNIST and Fashion-MNIST as the datasets, and MLP and CNN as the ML models to test the accuracy of the ML model in our proposed scheme. We vary the size of nodes' assigned local datasets from 2 Mbits to 8 Mbits and set the learning rate $r = 0.1$. In addition, we set the number of T rounds within one cycle to 5 rounds and the total number of rounds of global iteration to 100. This means that nodes' locally trained models are aggregated to generate an updated global model after 5 iteration rounds. The newly generated global model is then broadcasted to each node, which uses it to conduct the next cycle of iteration. The results in Fig. 7 demonstrate that our proposed scheme can achieve satisfactory convergence results on both the MNIST and Fashion-MNIST datasets. This validates the effectiveness of our proposed approach. Additionally, we can observe from Fig. 7 that a larger size of the local dataset leads to a faster convergence rate and higher accuracy.

Fig. 8 compares our proposed scheme with the pFedMe proposed in [41] in terms of the accuracy of the model with malicious users. We consider malicious nodes conducting collusion behaviors during model verification and trust

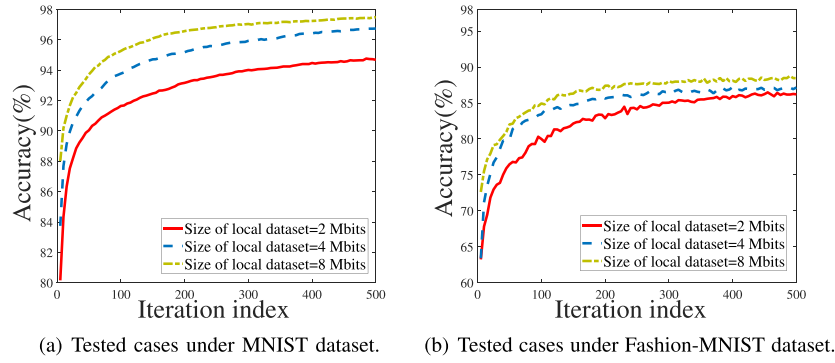


Fig. 7. The accuracy of the models trained on nodes' assigned local datasets of various sizes under MNIST and Fashion-MNIST dataset.

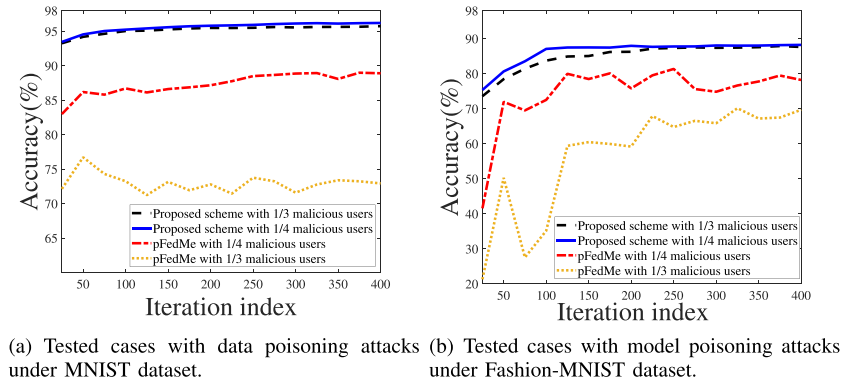


Fig. 8. Performance comparison in terms of the accuracy of the models with malicious users.

TABLE II
THREATS MODELS RESISTED BY THE PROTOCOLS

Security properties	Su et al. [36]	Wang et al. [9]	Yang et al. [42]	Lv et al. [25]	Putra et al. [33]	Our proposed scheme
Resistance to poisoning attack	✓	✓	✓	✓	-	✓
Resistance to historical behaviors tampering attack	✓	✓	✓	✓	✓	✓
Resistance to sybil attack	✓	✓	-	✗	✓	✓
Resistance to newcomer attack	✓	✓	-	✗	✓	✓
Resistance to self-promoting attack	-	-	-	-	✓	✓
Resistance to bad-mouthing attack	-	-	✓	-	✓	✓
Resistance to ballot-stuffing attack	-	-	✓	-	✓	✓
Resistance to free-riding attack	✓	✗	-	✓	-	✓

calculation via ballot-stuffing attacks or bad-mouthing attacks. Besides, the malicious nodes also conduct model poisoning attacks through Gaussian noise injection and data poisoning attacks via label-flipping, respectively. Here, we utilize the MNIST and Fashion-MNIST datasets, employing CNN and MLP models to evaluate the accuracy of ML models. We set

the number of T rounds within one cycle to 25 rounds and the overall global iteration to 16 rounds. The results in Fig. 8 show that our proposed scheme can achieve better performance than the pFedMe with malicious users. Our proposed scheme can effectively detect and expel malicious nodes that engage in collusion attacks and poisoning attacks, preventing them from

affecting positively behaving nodes and the system's overall performance. Regarding pFedMe, the absence of filtering processes and essential detection or punishment mechanisms for malicious nodes makes it ineffective against collusion attacks and poisoning attacks. This reduces the number of nodes with positive behavior in the computation group and degrades overall system performance. Besides, pFedMe uses a fixed number of local iterations, which incurs higher system costs and a lower utility compared to our scheme according to Fig. 4.

D. Reliability and Security Analysis

In our proposed scheme, the following security threats are considered and effectively addressed.

- *Illegal access:* As we mentioned above, malicious nodes can falsify their identities to carry out illegal access, such as through sybil or newcomer attacks. To tackle these issues, we design a blockchain assisted trust management scheme. Each node is required to register its intrinsic properties and attributes to the server, which ensures a one-to-one mapping relationship between digital identity and physical entity. The server then stores the registration messages in the attribute blockchain network, with the goal of preventing being tampered with. The attribute blockchain network can track the identity information of nodes, ensuring that each node uses only one identity for registration. Besides, illegal access attempts may degrade a node's trust value and utility, leading to its expulsion from the computation group. Based on these manipulations, sybil attacks and newcomer attacks can be effectively avoided.
- *Data and model poisoning:* The malicious nodes may conduct Gaussian noise injection or label-flipping to degrade the performance of their locally trained model. To tackle these challenges, the filtering process is proposed in Algorithm 1 to identify the malicious nodes with abnormal local models. Moreover, in our proposed trust management scheme, the malicious nodes with abnormal local models will receive poor evaluation results from other nodes, resulting in poor trust values, corresponding punishment, and less chance of participating in the computation group to degrade the performance of the global model. As shown in Fig. 8, our proposed scheme achieves satisfactory model convergence on both the MNIST and Fashion-MNIST datasets under poisoning attacks, which shows its robustness against both data and model poisoning attacks.
- *Data tampering:* Apart from the poisoning attacks, the malicious nodes may tamper with the historical record of their bad behaviors to obtain the whitewashed trust values. In our proposed scheme, the historical records of nodes' behaviors are packed and stored in the main blockchain by the server. If the malicious nodes attempt to tamper with the historical record of their behaviors in the submitted block, they need to obtain at least 2/3 agreements of the nodes within the computation group, which is unlikely and much more costly under the assumption that the proportion of malicious nodes among all nodes does not exceed 1/3 [36].

Thus, historical behaviors tampering attacks can be resisted with high probability.

- *Cheating in the trust calculation process:* During the trust calculation process, some nodes may conduct malicious behaviors, such as bad-mouthing attacks, self-promoting attacks, and ballot-stuffing attacks, with the goal of obtaining exaggerated trust values or reduce the chances of other users joining the computation group. In our proposed scheme, nodes broadcast their locally trained models to other nodes for evaluation. The average value of node i 's quality value and other nodes' evaluations for node i serves as the baseline. In addition, we introduce a tolerable bias between the average value and node i 's quality value and other nodes' evaluations for node i to detect the abnormal value. As we assume that the number of malicious nodes and the value range of the nodes' quality values are limited, it is unlikely that abnormal values from malicious nodes will significantly affect the average value. Thus, the abnormal value of the nodes' quality values and evaluations can be detected with high probability. Moreover, in our proposed scheme, each node is required to attach its signature to its quality value and the evaluations to other nodes, enabling effective tracing of cheating nodes during trust calculation. These nodes will receive low trust values and corresponding punishments. Therefore, cheating in the trust calculation process can be tackled.
- *Free-riding attack:* Some nodes may be reluctant to invest effort into improving the performance of their local ML models and trust values due to concerns about energy and latency consumption, resulting in the dissemination of low-quality models and decreasing the performance of the global model generated by the final global aggregation. To effectively solve this problem, we introduce an incentive mechanism to motivate these nodes to improve the quality of their locally trained ML models and earn corresponding rewards. Furthermore, the performance of each node's local ML model is considered a part of its trust value and serves as a criterion for participating in the computation group. As we can observe in Fig. 6, these nodes with low-quality models are being punished and kicked out from the computation group, which thus effectively prevents the free-riding attacks.

As shown in Table II, relevant blockchain assisted trust management schemes [9], [25], [33], [36], [42] are listed to compare the security properties with our proposed scheme.

VIII. CONCLUSION

In this article, we have focused on the data-parallel distributed learning where MTs' datasets are aggregated to create a main dataset. This main dataset is sent to the edge layer, split into sub-datasets, and distributed to each edge-layer node with a pre-trained model for local training. The trust values, determining nodes' eligibility to participate in the computation group, is calculated based on the quality of their local models, evaluations by other nodes, and the weighted sum of their historical behaviors. Blockchain assisted trust management scheme using a PoS

mechanism has been designed to enable continuous evaluation of edge-layer nodes' model training and establishes a dynamic join and exit mechanism for the nodes. This scheme includes nodes' attributes registration, trust calculation, information saving, and block writing. The nodes' trust values serve as the stakes for PoS mechanism to enable efficient consensus among the nodes. Additionally, we employed the MaxClique method-based filtering process to identify and filter out nodes engaged in poisoning attacks. To motivate the nodes to invest effort in the learning process while discouraging any malicious behaviors, we have designed an incentive mechanism and investigated dynamic optimization for the node's local iterations and the threshold of the trust value to maximize the overall utility of the nodes and enhance the system performance and security. A PPO based algorithm has been designed to effectively solve the optimization problem and obtain the optimal strategy of nodes' local iterations and selection, achieving the trade-off between efficiency and quality of distributed learning. Experimental results and security analysis have been provided to show the performance advantages of our proposed scheme under various parameter configurations, as well as its capability to effectively address common security threats.

For the future work, we plan to enhance data privacy protection during transmission, say, using differential privacy techniques, and explore the balance between data privacy, learning efficiency, and performance.

REFERENCES

- [1] S. Lee, J. Lee, and K. Lee, "DeepVehicleSense: An energy-efficient transportation mode recognition leveraging staged deep learning over sound samples," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3270–3286, Jun. 2023.
- [2] P. Lang, D. Tian, X. Duan, J. Zhou, Z. Sheng, and V. C. M. Leung, "Cooperative computation offloading in blockchain-based vehicular edge computing networks," *IEEE Trans. Intell. Veh.*, vol. 7, no. 3, pp. 783–798, Sep. 2022.
- [3] P. Lang, D. Tian, X. Duan, J. Zhou, Z. Sheng, and V. C. M. Leung, "Blockchain-based cooperative computation offloading and secure handover in vehicular edge computing networks," *IEEE Trans. Intell. Veh.*, vol. 8, no. 7, pp. 3839–3853, Jul. 2023.
- [4] J. Verbraken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeyer, "A survey on distributed machine learning," *ACM Comput. Surv.*, vol. 53, no. 2, pp. 1–33, 2020.
- [5] C. Nagaraju, Y. Ramesh, and C. K. Mohan, "A data parallel approach for distributed neural networks to achieve faster convergence," in *Proc. 16th Int. Conf. Mach. Vis.*, 2024, pp. 380–389.
- [6] E. Xing, Q. Ho, P. Xie, and D. Wei, "Strategies and principles of distributed machine learning on Big Data," *Engineering*, vol. 2, pp. 179–195, 2016.
- [7] P. M. S. Sánchez et al., "Studying the robustness of anti-adversarial federated learning models detecting cyberattacks in IoT spectrum sensors," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 573–584, Mar./Apr. 2024.
- [8] J. Guo et al., "TFL-DT: A trust evaluation scheme for federated learning in digital twin for mobile networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3548–3560, Nov. 2023.
- [9] P. Wang, N. Xu, H. Zhang, W. Sun, and A. Benslimane, "Dynamic access control and trust management for blockchain-empowered IoT," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 12997–13009, Aug. 2022.
- [10] M. Dai, Z. Su, Q. Xu, Y. Wang, and N. Lu, "A trust-driven contract incentive scheme for mobile crowd-sensing networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 2, pp. 1794–1806, Feb. 2022.
- [11] Y. Du et al., "Blockchain-aided edge computing market: Smart contract and consensus mechanisms," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3193–3208, Jun. 2023.
- [12] X. Hao, W. Ren, Y. Fei, T. Zhu, and K. K. R. Choo, "A blockchain-based cross-domain and autonomous access control scheme for Internet of Things," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 773–786, Mar./Apr. 2023.
- [13] C. T. Nguyen et al., "BlockRoam: Blockchain-based roaming management system for future mobile networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 3880–3894, Nov. 2022.
- [14] A. Agarwal, S. M. Kakade, J. D. Lee, and G. Mahajan, "Optimality and approximation with policy gradient methods in Markov decision processes," in *Proc. 33rd Conf. Learn. Theory*, 2020, pp. 64–66.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv: 1707.06347*.
- [16] M. Goudarzi, M. Palaniswami, and R. Buyya, "A distributed deep reinforcement learning technique for application placement in edge and fog computing environments," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2491–2505, May 2023.
- [17] Y. Qu, S. R. Pokhrel, S. Garg, L. Gao, and Y. Xiang, "A blockchained federated learning framework for cognitive computing in industry 4.0 networks," *IEEE Trans. Ind. Inform.*, vol. 17, no. 4, pp. 2964–2973, Apr. 2021.
- [18] M. A. P. Putra, R. N. Alief, S. M. Rachmawati, G. A. Sampedro, D. S. Kim, and J. M. Lee, "Proof-of-authority-based secure and efficient aggregation with differential privacy for federated learning in Industrial IoT," *Internet Things*, vol. 25, 2024, Art. no. 101107.
- [19] L. Jiang, Y. Liu, H. Tian, L. Tang, and S. Xie, "Resource-efficient federated learning and DAG blockchain with sharding in digital-twin-driven industrial IoT," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17113–17127, May 2024.
- [20] C. Xu, Y. Qu, T. H. Luan, P. W. Eklund, Y. Xiang, and L. Gao, "An efficient and reliable asynchronous federated learning scheme for smart public transportation," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6584–6598, May 2023.
- [21] S. Zhu, R. Li, Z. Cai, D. Kim, D. Seo, and W. Li, "Secure verifiable aggregation for blockchain-based federated averaging," *High-Confidence Comput.*, vol. 2, no. 1, 2022, Art. no. 100046.
- [22] C. Zhang et al., "A blockchain-based model migration approach for secure and sustainable federated learning in IoT systems," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6574–6585, Apr. 2023.
- [23] M. Tang, F. Peng, and V. W. S. Wong, "A blockchain-empowered incentive mechanism for cross-silo federated learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9240–9253, Oct. 2024.
- [24] A. Gholami, N. Torkzaban, and J. S. Baras, "Trusted decentralized federated learning," in *Proc. 19th IEEE Annu. Consum. Commun. Netw. Conf.*, Las Vegas, NV, USA, 2022, pp. 1–6.
- [25] P. Lv, L. Xie, J. Xu, X. Wu, and T. Li, "Misbehavior detection in vehicular ad hoc networks based on privacy-preserving federated learning and blockchain," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 3936–3948, Dec. 2022.
- [26] J. Qi, F. Lin, Z. Chen, C. Tang, R. Jia, and M. Li, "High-quality model aggregation for blockchain-based federated learning via reputation-motivated task participation," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18378–18391, Oct. 2022.
- [27] Y. Wu, Y. Song, T. Wang, L. Qian, and T. Q. S. Quek, "Non-orthogonal multiple access assisted federated learning via wireless power transfer: A cost-efficient approach," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2853–2869, Apr. 2022.
- [28] K. Wei, J. Li, M. Ding, C. Ma, Y. -S. Jeon, and H. V. Poor, "Covert model poisoning against federated learning: Algorithm design and optimization," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1196–1209, May/Jun. 2024.
- [29] T. Wang, B. Sun, L. Wang, X. Zheng, and W. Jia, "EIDLS: An edge-intelligence-based distributed learning system over Internet of Things," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 53, no. 7, pp. 3966–3978, Jul. 2023.
- [30] H. Chen and F. Koushanfar, "Tutorial: Toward robust deep learning against poisoning attacks," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 3, pp. 1–15, 2023.
- [31] M. H. u. Rehman, A. M. Dirir, K. Salah, E. Damiani, and D. Svetinovic, "TrustFed: A framework for fair and trustworthy cross-device federated learning in IIoT," *IEEE Trans. Ind. Inform.*, vol. 17, no. 12, pp. 8485–8494, Dec. 2021.

- [32] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Trans. Dependable Secure Comput.*, vol. 12, no. 5, pp. 519–532, Sep./Oct. 2015.
- [33] G. D. Putra, V. Dedeoglu, S. S. Kanhere, R. Jurdak, and A. Ignjatovic, "Trust-based blockchain authorization for IoT," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1646–1658, Jun. 2021.
- [34] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1935–1949, Mar. 2021.
- [35] D. Cao, S. Chang, Z. Lin, G. Liu, and D. Sun, "Understanding distributed poisoning attack in federated learning," in *Proc. IEEE 25th Int. Conf. Parallel Distrib. Syst.*, Tianjin, China, 2019, pp. 233–239.
- [36] Z. Su, Y. Wang, Q. Xu, and N. Zhang, "LVBS: Lightweight vehicular blockchain for secure data sharing in disaster rescue," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 19–32, Jan./Feb. 2022.
- [37] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, and W. Luo, "DeepChain: Auditable and privacy-preserving deep learning with blockchain-based incentive," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2438–2455, Sep./Oct. 2021.
- [38] M. Samir, D. Ebrahimi, C. Assi, S. Sharafeddine, and A. Ghayeb, "Leveraging UAVs for coverage in cell-free vehicular networks: A deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 9, pp. 2835–2847, Sep. 2021.
- [39] I. Grondman, L. Busoniu, G. A. D. Lopes, and R. Babuska, "A survey of actor-critic reinforcement learning: Standard and natural policy gradients," *IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.)*, vol. 42, no. 6, pp. 1291–1307, Nov. 2012.
- [40] F. Yu, H. Lin, X. Wang, A. Yassine, and M. S. Hossain, "Blockchain-empowered secure federated learning system: Architecture and applications," *Comput. Commun.*, vol. 196, pp. 55–65, 2022.
- [41] C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with Moreau envelopes," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21394–21405.
- [42] Z. Yang, K. Yang, L. Lei, K. Zheng, and V. C. M. Leung, "Blockchain-based decentralized trust management in vehicular networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1495–1505, Apr. 2019.



Yuxiao Song received the BSc degree in software engineering from East China Normal University, in 2020, and the MS degree in computer science from the University of Macau, in 2022. He is currently working toward the PhD degree in School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. His research interests focus on security in distributed learning.



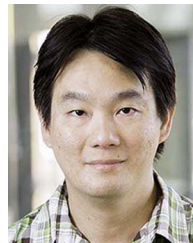
Daojing He (Member, IEEE) received the BEng and MEng degrees from the Harbin Institute of Technology, China, in 2007 and 2009, respectively, and the PhD degree from Zhejiang University, China, in 2012, all in computer science. He is currently a professor in the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, China. His research interests include network and systems security. He is on the editorial board of some international journals.



Minghui Dai received the PhD degree from Shanghai University, China, in 2021. He is currently with the School of Computer Science and Technology, Donghua University. His research interests are in the general area of wireless network architecture and vehicular networks.



Sammy Chan (Senior Member, IEEE) received the BE and MEngSc degrees in electrical engineering from the University of Melbourne, Australia, in 1988 and 1990, respectively, and the PhD degree in communication engineering from the Royal Melbourne Institute of Technology, Australia, in 1995. Since Dec. 1994 he has been with the Department of Electrical Engineering, City University of Hong Kong, where he is currently an associate professor.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the PhD degree in information security from the Queensland University of Technology, Australia, in 2006. He currently holds the Cloud Technology Endowed Professorship with the University of Texas at San Antonio. He is the founding co-editor-in-chief of ACM Distributed Ledger Technologies: Research and Practice, and the founding chair of IEEE Technology and Engineering Management Society Technical Committee (TC) on Blockchain and Distributed Ledger Technologies. He is the recipient of the 2022 IEEE Hyper-Intelligence TC Award for Excellence in Hyper-Intelligence Systems (Technical Achievement award), the 2022 IEEE TC on Homeland Security Research and Innovation Award, the 2022 IEEE TC on Secure and Dependable Measurement Mid-Career Award, and the 2019 IEEE TC on Scalable Computing Award for Excellence in Scalable Computing (Middle Career Researcher).



Mohsen Guizani (Fellow, IEEE) is currently a professor of machine learning with the Mohamed Bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates. His research interests include applied machine learning, artificial intelligence, Internet of Things, smart city, and cybersecurity. He was elevated to the IEEE Fellow, in 2009 and was listed as a Clarivate Analytics Highly Cited Researcher in Computer Science since 2019.